

Assignment 9: ChefPalette — Recipe Sharing API Pending



Objective

Build a **FastAPI-based Recipe Sharing API** where users can create recipes, search recipes by ingredients, like recipes, view trending recipes, and update their own recipe details.

The assignment should demonstrate:

- List-based Pydantic Models
- Dependency Injection for Database Sessions
- Complex Query Filters
- Partial Matching
- Like Count Aggregation
- Validation and Error Handling

Technical Requirements

- **Framework:** FastAPI
- Use dependency injection for database sessions
- Ingredients must be stored and handled as a **list of strings**
- Ingredient search must support **partial matching**
- Duplicate likes from the same user must be prevented
- Trending recipes must be sorted by **like count in descending order**
- Support optional filtering using a `prep_time` range

Data Models

User

- id
- email

Recipe

- id
- title
- ingredients
- instructions
- prep_time
- user_id

Like

- id
- user_id
- recipe_id

API Routes

POST /recipes

Create a new recipe.

Required fields:

- Title
- Ingredients
- Instructions
- Prep time

Ingredients must be provided as a **list of strings**.

Example:

```
{  
  "title": "Vegetable Pasta"
```

```
title: "Vegetable Pasta",
"ingredients": [
  "pasta",
  "tomato",
  "onion",
  "cheese"
],
"instructions": "Cook the pasta and mix with prepared vegetables.",
"prep_time": 30
}
```

Validation:

- Ingredients list cannot be empty.
 - `prep_time > 0`
-

GET /recipes

Search and filter recipes.

Support optional query parameters such as:

- `ingredient`
- `min_prep_time`
- `max_prep_time`

Example:

`/recipes?ingredient=tomato`

Ingredient search must support **partial matching**.

For example, searching for:

tom

should match an ingredient such as:

tomato

Prep-time filtering example:

`/recipes?min_prep_time=10&max_prep_time=30`

GET /recipes/trending

Return the top recipes based on the number of likes.

Recipes must be sorted:

Highest Likes → Lowest Likes

The response should include the recipe details and corresponding like count.

POST /recipes/{id}/like

Allow the current user to like a recipe.

Rules:

- A user can like a recipe only once.
- Duplicate likes from the same user must return:

400 Bad Request

- Return 404 Not Found if the recipe does not exist.
-

PUT /recipes/{id}

Update an existing recipe.

Allow updates to:

- Ingredients
- Instructions

Validation rules must still apply when ingredients are updated.

Return 404 Not Found if the recipe does not exist.

List-Based Pydantic Validation

Ingredients must be implemented as a list.

Example:

```
ingredients: list[str]
```

The ingredients list must contain at least one item.

Example validation:

```
ingredients: list[str] = Field(min_length=1)
prep_time: int = Field(gt=0)
```

Database Dependency Injection

Use a database dependency such as:

```
Depends(get_db)
```

to provide database sessions to the API routes.

Database session creation and cleanup should be handled through the dependency.

Validation & Business Rules

- Ingredients list cannot be empty.
 - `prep_time` must be greater than `0`.
 - Ingredient search must support partial matching.
 - Duplicate likes from the same user are not allowed.
 - Trending recipes must be sorted by like count in descending order.
 - Recipe IDs must be validated before like or update operations.
 - Prep-time range filtering should be supported.
-

Error Handling

Handle appropriate errors for:

- Recipe not found
- Empty ingredients list
- Invalid prep time
- Duplicate like

- Invalid filter values
- Invalid request data

Use appropriate HTTP status codes such as 400, 404, and 422.

API Testing

All APIs should be testable through FastAPI Swagger:

/docs

Test:

- Recipe creation
 - Ingredients list validation
 - Ingredient partial search
 - Prep-time filtering
 - Recipe likes
 - Duplicate like validation
 - Trending recipe sorting
 - Recipe update
 - Database session dependency
-

Submission Instructions

Submit the **complete project as ONE ZIP file through the LMS.**

The ZIP file should include:

- Complete source code
- requirements.txt
- README.md
- All files required to run the project

ZIP File Name

StudentName_ChefPalette.zip

Example:

RahulKumar_ChefPalette.zip

Important

- Submit only **one ZIP file**.
 - Do not upload individual files separately.
 - Do not include `venv`, `.venv`, `__pycache__`, or unnecessary folders.
 - Test the application before submission.
 - Ensure the correct ZIP file is uploaded successfully to the LMS.
 - Submit before the deadline mentioned in the LMS.
-

Evaluation Criteria

The assignment will be evaluated based on:

- Correct API functionality
- List-based Pydantic model implementation
- Database dependency injection
- Ingredient partial matching
- Prep-time filtering
- Duplicate like prevention
- Trending recipe logic
- Validation and error handling
- Code quality and organization
- Successful project execution
- Proper ZIP submission

Expected Outcome

By completing this assignment, you should be able to build a structured **FastAPI Recipe Sharing API** using **list-based validation, database dependency injection, complex filters, partial search, likes, and trending logic**.

Upload Assignment